

Sorting Algorithms Programs in C

3. Bubble Sort

```
#include<stdio.h>
#include<conio.h>
void main(){
    int a[100], n, i, j, temp;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    for(i=0; i<n-1; i++){
        for(j=0; j<n-i-1; j++){
            if(a[j] > a[j+1]){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
    printf("Sorted elements:\n");
    for(i=0; i<n; i++)
        printf("%d ", a[i]);
    getch();
}
```

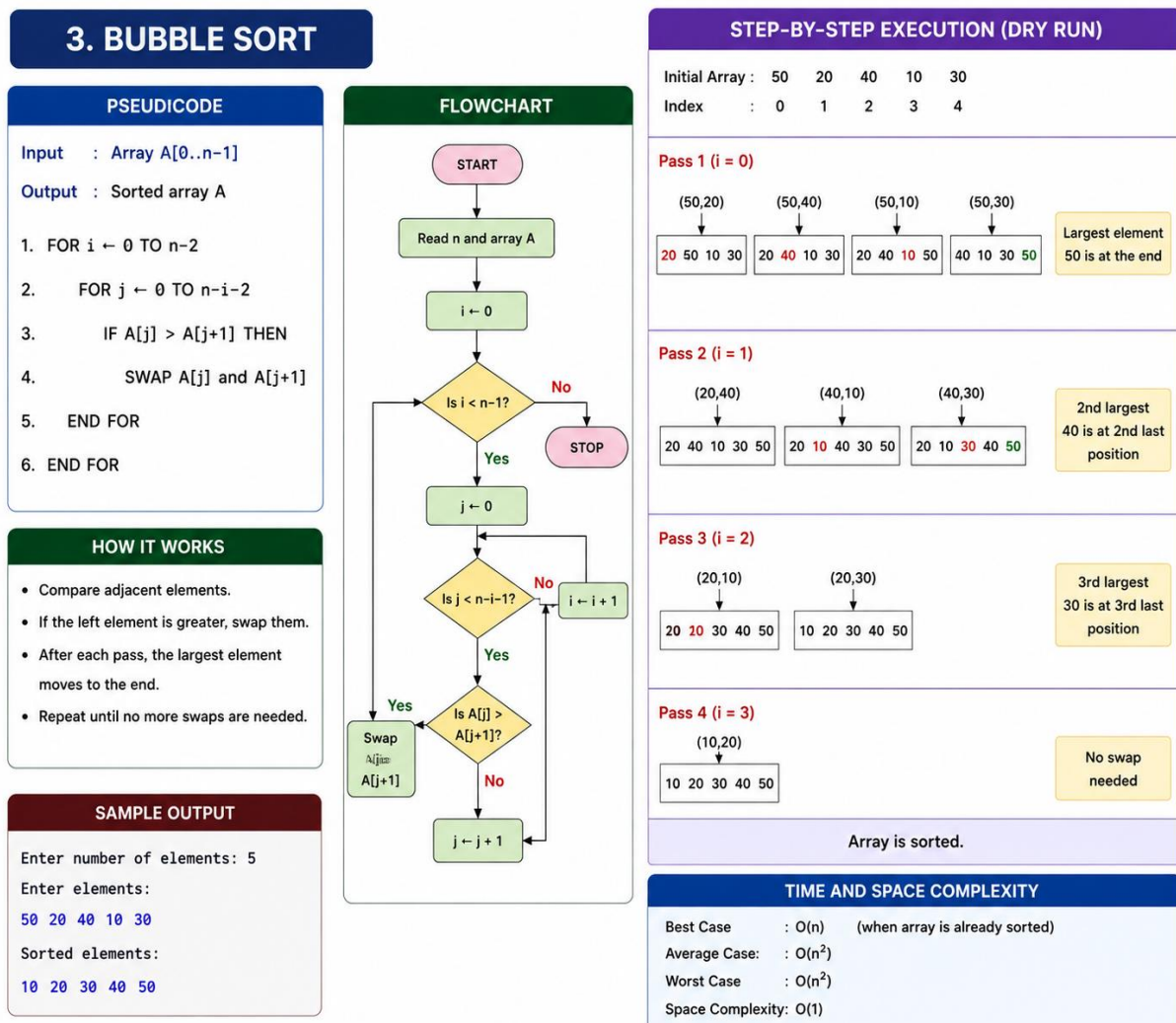
Output: Enter number of elements: 5

Enter elements:

50 20 40 10 30

Sorted elements:

10 20 30 40 50



```

#include<stdio.h>
#include<conio.h>
void main(){
    int a[100], n, i, j, min, temp;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    for(i=0; i<n-1; i++){
        min = i;
        for(j=i+1; j<n; j++){
            if(a[j] < a[min])
                min = j;
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
    printf("Sorted elements:\n");
    for(i=0; i<n; i++)
        printf("%d ", a[i]);
    getch();
}

```

Output: Enter number of elements: 5

Enter elements:

64 25 12 22 11

Sorted elements:

11 12 22 25 64

4. SELECTION SORT

PSEUDOCODE

Input : Array A[0..n-1]

Output : Sorted array A

```
1. FOR i ← 0 TO n-2
2.   min ← i
3.   FOR j ← i+1 TO n-1
4.     IF A[j] < A[min] THEN
5.       min ← j
6.   IF min ≠ i THEN
7.     SWAP A[i] and A[min]
8. END FOR
```

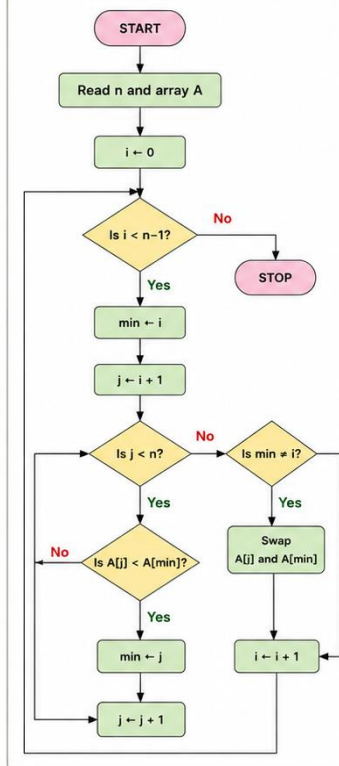
HOW IT WORKS

- Find the smallest element in the unsorted part.
- Swap it with the first unsorted element.
- Move the boundary of unsorted array one position forward.
- Repeat until the entire array is sorted.

SAMPLE OUTPUT

```
Enter number of elements: 5
Enter elements:
64 25 12 22 11
Sorted elements:
11 12 22 25 64
```

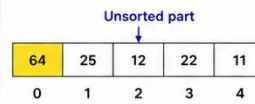
FLOWCHART



STEP-BY-STEP EXECUTION (DRY RUN)

Initial Array : 64 25 12 22 11
Index : 0 1 2 3 4

i = 0



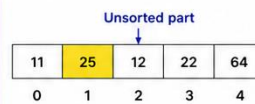
Smallest in unsorted part is 11
(at index 4)

Swap A[0] and A[4]

Array after swap:

11 25 12 22 64

i = 1



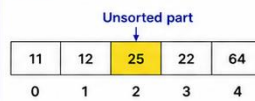
Smallest in unsorted part is 12
(at index 2)

Swap A[1] and A[2]

Array after swap:

11 12 25 22 64

i = 2



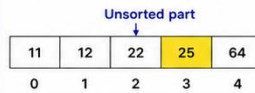
Smallest in unsorted part is 22
(at index 3)

Swap A[2] and A[3]

Array after swap:

11 12 22 25 64

i = 3



Smallest in unsorted part is 25
(at index 3)

Already in correct position

No swap needed

Array is sorted.

TIME AND SPACE COMPLEXITY

Best Case : $O(n^2)$ (when array is already sorted)
Average Case : $O(n^2)$
Worst Case : $O(n^2)$
Space Complexity : $O(1)$

5. Insertion Sort, Iterative

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main(){
```

```
    int a[100], n, i, j, key;
```

```
    clrscr();
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter elements:\n");
```

```
    for(i=0;i<n;i++){
```

```
        scanf("%d",&a[i]);
```

```
    for(i=1;i<n;i++){
```

```
        key=a[i];
```

```
        j=i-1;
```

```
        while(j>=0 && a[j]>key){
```

```
            a[j+1]=a[j];
```

```

        j--;
    }
    a[j+1]=key;
}
printf("Sorted elements:\n");
for(i=0;i<n;i++)
    printf("%d ",a[i]);
getch();
}

```

Output: Enter number of elements: 5

Enter elements:

12 11 13 5 6

Sorted elements:

5 6 11 12 13

5. INSERTION SORT (ITERATIVE)

PSEUDOCODE

Input : Array A[0..n-1]
Output : Sorted array A

- FOR $i \leftarrow 1$ TO $n-1$
- key $\leftarrow A[i]$
- $j \leftarrow i - 1$
- WHILE $j \geq 0$ AND $A[j] > \text{key}$
- $A[j + 1] \leftarrow A[j]$
- $j \leftarrow j - 1$
- END WHILE
- $A[j + 1] \leftarrow \text{key}$
- END FOR

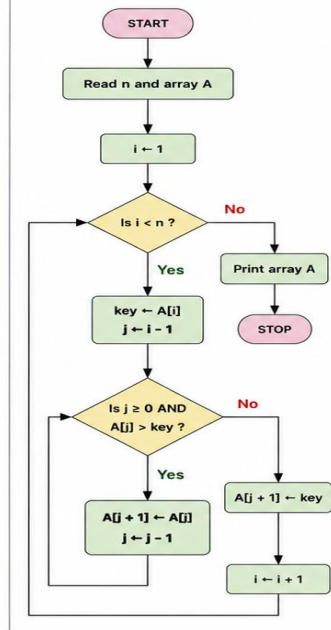
HOW IT WORKS

- Start from the second element.
- Compare the key (current element) with elements in the sorted left part.
- Shift greater elements one position ahead.
- Insert the key in its correct position.
- Repeat for all elements.

SAMPLE OUTPUT

Enter number of elements: 5
Enter elements:
12 11 13 5 6
Sorted elements:
5 6 11 12 13

FLOWCHART



STEP-BY-STEP EXECUTION (DRY RUN)

Initial Array : 12 11 13 5 6
Index : 0 1 2 3 4

Step 1: i = 1, key = 11

Compare 11 with 12 → $11 < 12$
Shift 12 to right
Insert 11 at position 0

11	12	13	5	6
0	1	2	3	4

Step 2: i = 2, key = 13

Compare 13 with 12 → $13 > 12$
No shift needed
Insert 13 at position 2

11	12	13	5	6
0	1	2	3	4

Step 3: i = 3, key = 5

Compare 5 with 13 → $5 < 13$ (shift)
Compare 5 with 12 → $5 < 12$ (shift)
Compare 5 with 11 → $5 < 11$ (shift)
Insert 5 at position 0

5	11	12	13	6
0	1	2	3	4

Step 4: i = 4, key = 6

Compare 6 with 13 → $6 < 13$ (shift)
Compare 6 with 12 → $6 < 12$ (shift)
Compare 6 with 11 → $6 < 11$ (shift)
Compare 6 with 5 → $6 > 5$ (stop)
Insert 6 at position 1

5	6	11	12	13
0	1	2	3	4

Array is sorted.

TIME AND SPACE COMPLEXITY

Best Case : $O(n)$ (array already sorted)
Average Case : $O(n^2)$
Worst Case : $O(n^2)$ (array in reverse order)
Space Complexity : $O(1)$ (in-place sorting)

6. Shell Sort, Iterative

```

#include<stdio.h>
#include<conio.h>
void main(){
    int a[100], n, i, j, gap, temp;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)

```

```

scanf("%d",&a[i]);
for(gap=n/2; gap>0; gap=gap/2){
    for(i=gap; i<n; i++){
        temp=a[i];
        for(j=i; j>=gap && a[j-gap]>temp; j=j-gap){
            a[j]=a[j-gap];
        }
        a[j]=temp;
    }
}
printf("Sorted elements:\n");
for(i=0; i<n; i++)
    printf("%d ",a[i]);
getch();
}

```

Output

Enter number of elements: 5

Enter elements:

45 23 12 89 1

Sorted elements:

1 12 23 45 89

6. SHELL SORT (ITERATIVE)

PSEUDOCODE

Input : Array A[0..n-1]
Output : Sorted array A

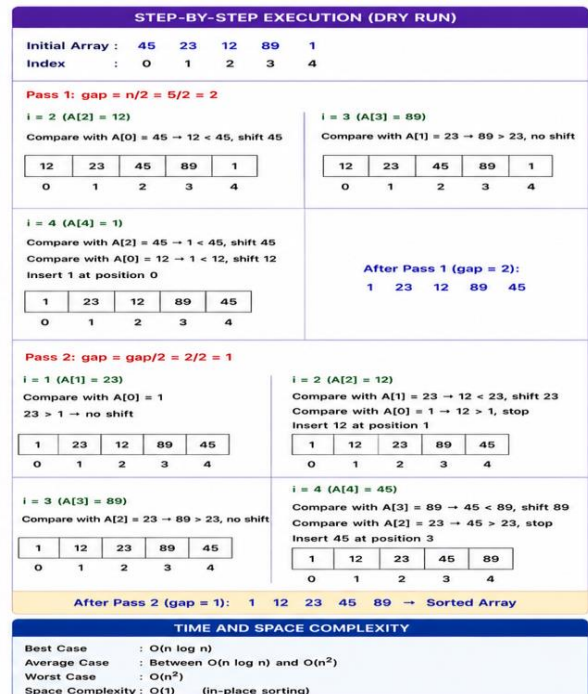
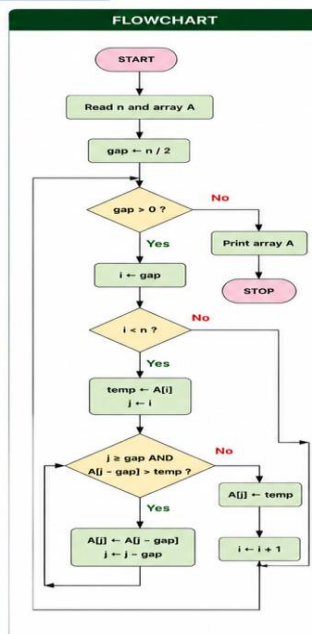
- gap $\leftarrow n / 2$
- WHILE gap > 0
- FOR i $\leftarrow$ gap TO n-1
- temp $\leftarrow A[i]$
- j $\leftarrow i$
- WHILE j $\geq$ gap AND A[j-gap] > temp
- A[j] $\leftarrow A[j - gap]$
- j $\leftarrow j - gap$
- END WHILE
- A[j] $\leftarrow temp$
- END FOR
- gap $\leftarrow gap / 2$
- END WHILE

HOW IT WORKS

- Start with a large gap (n/2).
- Perform a gapped insertion sort.
- Reduce the gap (gap = gap/2).
- Repeat until gap becomes 0.
- When gap = 1, it becomes a normal insertion sort.
- Array is sorted.

SAMPLE OUTPUT

Enter number of elements: 5
Enter elements:
45 23 12 89 1
Sorted elements:
1 12 23 45 89



7. Merge Sort

```
#include<stdio.h>
```

```

#include<conio.h>

int a[50];

void merge(int low,int mid,int high){
    int h,i,j,b[50],k;

    h=low;

    i=low;

    j=mid+1;

    while((h<=mid)&&(j<=high)){

        if(a[h]<=a[j]){

            b[i]=a[h];

            h++;

        }

        else{

            b[i]=a[j];

            j++;

        }

        i++;

    }

    if(h>mid){

        for(k=j;k<=high;k++){

            b[i]=a[k];

            i++;

        }

    }

    else{

        for(k=h;k<=mid;k++){

            b[i]=a[k];

            i++;

        }

    }

    for(k=low;k<=high;k++)

```

```

        a[k]=b[k];
    }
void mergesort(int low,int high){
    int mid;
    if(low<high){
        mid=(low+high)/2;
        mergesort(low,mid);
        mergesort(mid+1,high);
        merge(low,mid,high);
    }
}

void main(){
    int n,i;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    mergesort(0,n-1);
    printf("Sorted elements:\n");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    getch();
}

```

Output: Enter number of elements: 5

Enter elements:

50 20 40 10 30

Sorted elements:

10 20 30 40 50

7. MERGE SORT

PSEUDOCODE

Input : Array A[0..n-1]
Output : Sorted array A

MERGE-SORT(A, low, high)

```
1. if low < high
2.   mid ← (low + high) / 2
3.   MERGE-SORT(A, low, mid)
4.   MERGE-SORT(A, mid+1, high)
5.   MERGE(A, low, mid, high)
```

MERGE(A, low, mid, high)

```
1. i ← low, j ← mid+1, k ← low
2. while i ≤ mid and j ≤ high
3.   if A[i] ≤ A[j]
4.     temp[k] ← A[i]; i ← i + 1
5.   else
6.     temp[k] ← A[j]; j ← j + 1
7.   k ← k + 1
8. while i ≤ mid
9.   temp[k] ← A[i]; i ← i + 1; k ← k + 1
10. while j ≤ high
11.   temp[k] ← A[j]; j ← j + 1; k ← k + 1
12. for t ← low to high
13.   A[t] ← temp[t]
```

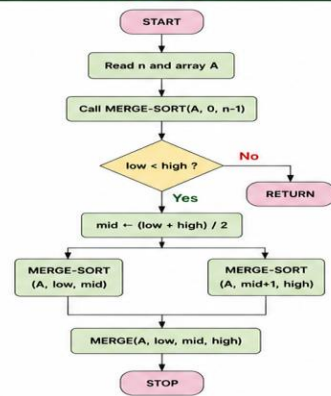
HOW IT WORKS

- Divide the array into two halves.
- Recursively sort both halves.
- Merge the two sorted halves.
- Continue until the whole array is sorted.

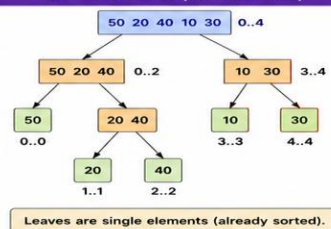
SAMPLE OUTPUT

Enter number of elements: 5
Enter elements:
50 20 40 10 30
Sorted elements:
10 20 30 40 50

FLOWCHART

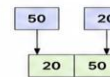


RECURSION TREE (DIVIDE PHASE)

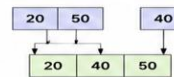


MERGE PHASE (STEP-BY-STEP)

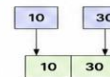
Step 1: Merge [50] and [20]



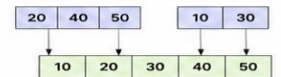
Step 2: Merge [20 50] and [40]



Step 3: Merge [10] and [30]



Step 4: Merge [20 40 50] and [10 30]



Step 4 Detailed Merge:

Left Array	Right Array	Merged Array (temp)
20 40 50	10 30	
i → 20	j → 10	k →

1) Compare 20 and 10 → 10 is smaller → add 10

20 40 50	10 30	
i → 20	j → 30	

2) Compare 20 and 30 → 20 is smaller → add 20

20 40 50	10 20	- - -
i → 40	j → 30	

3) Compare 40 and 30 → 30 is smaller → add 30

40 50	10 20 30	- -
i → 40	j →	k → 3

4) Right array finished → copy remaining left

40 50	- 20 30 40 50	
	k → 5 (done)	

Final Sorted Array: 10 20 30 40 50

TIME AND SPACE COMPLEXITY

Best Case : $O(n \log n)$
Average Case : $O(n \log n)$
Worst Case : $O(n \log n)$
Space Complexity : $O(n)$ (extra space for temp array)

8. Quick Sort

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int partition(int a[],int low,int high){
```

```
    int pivot,i,j,temp;
```

```
    pivot=a[low];
```

```
    i=low+1;
```

```
    j=high;
```

```
    while(i<=j) {
```

```
        while(a[i]<=pivot && i<=high)
```

```
            i++;
```

```
        while(a[j]>pivot)
```

```
            j--;
```

```
        if(i<j){
```

```
            temp=a[i];
```

```
            a[i]=a[j];
```

```
            a[j]=temp;
```

```

    }
}
temp=a[low];
a[low]=a[j];
a[j]=temp;
return j;
}

void quicksort(int a[],int low,int high){
    int p;
    if(low<high){
        p=partition(a,low,high);
        quicksort(a,low,p-1);
        quicksort(a,p+1,high);
    }
}

void main(){
    int a[20],n,i;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    quicksort(a,0,n-1);
    printf("Sorted elements:\n");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    getch();
}

```

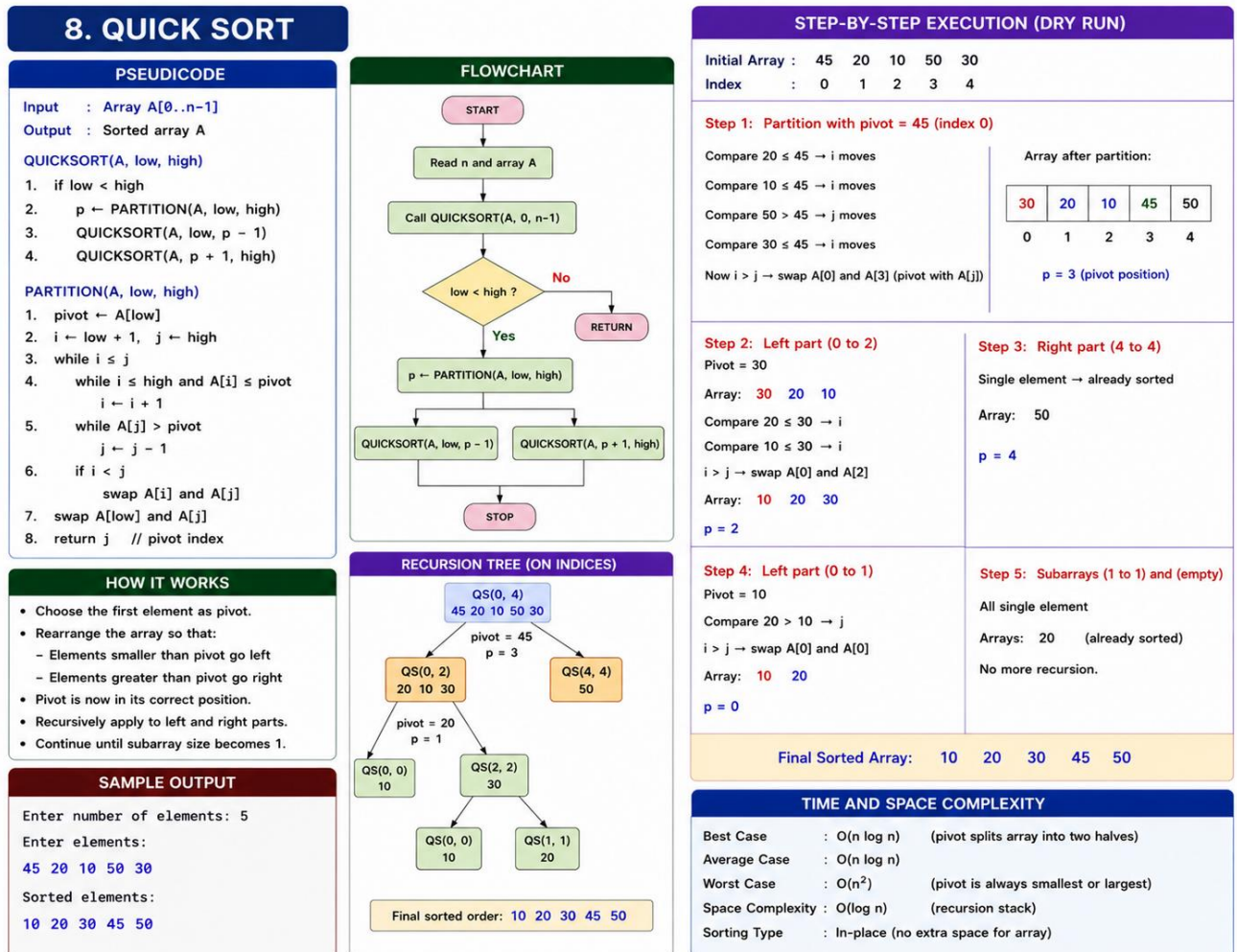
Enter number of elements: 5

Enter elements:

45 20 10 50 30

Sorted elements:

10 20 30 45 50



9. Heap Sort

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void heapify(int a[], int n, int i){
```

```
    int largest=i;
```

```
    int left=2*i+1;
```

```
    int right=2*i+2;
```

```

    int temp;
    if(left<n && a[left]>a[largest])
        largest=left;
    if(right<n && a[right]>a[largest])
        largest=right;
    if(largest!=i){
        temp=a[i];
        a[i]=a[largest];
        a[largest]=temp;
        heapify(a,n,largest);
    }
}

void heapsort(int a[], int n){
    int i, temp;
    for(i=n/2-1;i>=0;i--)
        heapify(a,n,i);
    for(i=n-1;i>=0;i--){
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;

        heapify(a,i,0);
    }
}

void main(){
    int a[20],n,i;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)

```

```

scanf("%d",&a[i]);

heapsort(a,n);

printf("Sorted elements:\n");

for(i=0;i<n;i++)

    printf("%d ",a[i]);

getch();

}

```

Enter number of elements: 5

Enter elements:

40 10 30 50 20

Sorted elements:

10 20 30 40 50

9. HEAP SORT

PSEUDOCODE

Input : Array A[0..n-1]
Output : Sorted array A

HEAPSORT(A, n)

1. Build Max Heap from A[0..n-1]
2. FOR i ← n-1 DOWNTO 1
3. swap A[0] and A[i]
4. heapify(A, i, 0)
5. END FOR

BUILD-MAX-HEAP(A, n)

1. FOR i ← n/2 - 1 DOWNTO 0
2. heapify(A, n, i)
3. END FOR

HEAPIFY(A, n, i)

1. largest ← i
2. l ← 2*i + 1 // left child
3. r ← 2*i + 2 // right child
4. IF l < n AND A[l] > A[largest]
5. largest ← l
5. IF r < n AND A[r] > A[largest]
6. largest ← r
6. IF largest ≠ i
7. swap A[i] and A[largest]
8. heapify(A, n, largest)

HOW IT WORKS

- Build a max heap from the array.
- The largest element will be at root (A[0]).
- Swap the root with last element.
- Reduce heap size by 1.
- Heapify the root.
- Repeat until heap size becomes 1.

SAMPLE OUTPUT

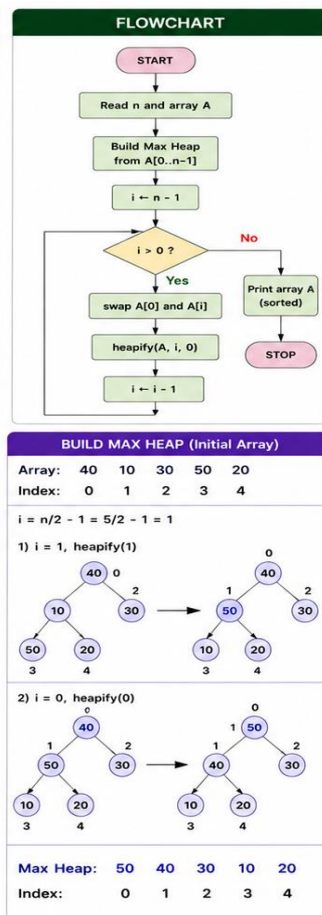
Enter number of elements: 5

Enter elements:

40 10 30 50 20

Sorted elements:

10 20 30 40 50



STEP-BY-STEP EXECUTION (DRY RUN)			
Initial Array : 40 10 30 50 20 Max Heap : 50 40 30 10 20			
Step	Operation	Heap (Unsorted Part)	Sorted Part
1	Swap A[0] and A[4] (heap size = 4) heapify(0)	20 40 30 10 50 0 1 2 3 4 40 20 30 10 50 0 1 2 3 4	50
2	Swap A[0] and A[3] (heap size = 3) heapify(0)	10 20 30 40 50 0 1 2 3 4 30 20 10 40 50 0 1 2 3 4	40 50
3	Swap A[0] and A[2] (heap size = 2) heapify(0)	10 20 30 40 50 0 1 2 3 4 20 10 30 40 50 0 1 2 3 4	30 40 50
4	Swap A[0] and A[1] (heap size = 1) heapify(0)	10 20 30 40 50 0 1 2 3 4 10 20 30 40 50 0 1 2 3 4	20 30 40 50
Final Sorted Array: 10 20 30 40 50			
TIME AND SPACE COMPLEXITY			
Build Max Heap	: O(n)		
Heapify (per call)	: O(log n)		
Total Time	: O(n log n)		
Best Case	: O(n log n)		
Average Case	: O(n log n)		
Worst Case	: O(n log n)		
Space Complexity	: O(1) (in-place sorting)		